```
    drop_table :distributors
  end
end
```

If your migration is irreversible, you should raise `ActiveRecord::IrreversibleMigration` from your `down` method. If someone tries to revert your migration, an error message will be displayed saying that it can't be done.

3.11 Reverting Previous Migrations

You can use Active Record's ability to rollback migrations using the `revert` method:

```
require_relative '2012121212_example_migration'

class FixupExampleMigration < ActiveRecord::Migration
  def change
    revert ExampleMigration

    create_table(:apples) do |t|
      t.string :variety
    end
  end
end
```

The `revert` method also accepts a block of instructions to reverse. This could be useful to revert selected parts of previous migrations. For example, let's imagine that `ExampleMigration` is committed and it is later decided it would be best to use Active Record validations, in place of the `CHECK` constraint, to verify the zipcode.

```
class DontUseConstraintForZipcodeValidationMigration <
ActiveRecord::Migration
  def change
    revert do
      # copy-pasted code from ExampleMigration
      reversible do |dir|
        dir.up do
          # add a CHECK constraint
          execute <<-SQL
            ALTER TABLE distributors
              ADD CONSTRAINT zipchk
                CHECK (char_length(zipcode) = 5);
          SQL
        end
        dir.down do
          execute <<-SQL
            ALTER TABLE distributors
              DROP CONSTRAINT zipchk
          SQL
        end
      end

      # The rest of the migration was ok
    end
  end
end
```

The same migration could also have been written without using `revert` but this would have involved a few more steps: reversing the order of `create_table` and `reversible`,
replacing `create_table`by `drop_table`, and finally replacing `up` by `down` and vice-versa. This is all taken care of by `revert`.

If you want to add check constraints like in the examples above, you will have to use `structure.sql` as dump method. See Schema Dumping and You.